

邊緣特徵提取與素描風格 即時轉換

Group 7

111360451 電子四甲 蕭賀澤

111310452 電機四甲 黃安華

專案動機與目標

研究動機：

現代影像濾鏡（如高斯模糊與色彩混疊）若由通用處理器 (CPU) 以純軟體執行，常受限於記憶體頻寬與指令集開銷，難以滿足即時處理之需求。

系統目標：

基於 **Zynq-7000 SoC** 架構，設計一具備高吞吐量 (**II=1**) 之專用硬體加速器，以減輕 **CPU** 負載並達成即時影像轉換。

演算法分析與設計

演算法演進與比較：

初期實作 (Sobel Edge Detection)：採用 3×3 梯度運算，運算成本低但影像輪廓生硬，缺乏光影細節。

改良實作 (Gaussian Blur & Color Dodge)：為達到平滑漸層之真實素描效果，改為 高斯模糊卷積，並透過色彩加亮演算法進行像素混疊 (Pixel Blending)。

Color Dodge 公式：
$$Result = \frac{A \times 256}{B + 1}$$
 (其中**A**是原圖、**B**是高斯模糊)

Sobel 演算法矩陣

X 方向邊緣偵測矩陣 (**G_x**)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Y 方向邊緣偵測矩陣 (**G_y**)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

高斯模糊矩陣

$$\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

矩陣權重總和 = 256

硬體微架構設計與最佳化

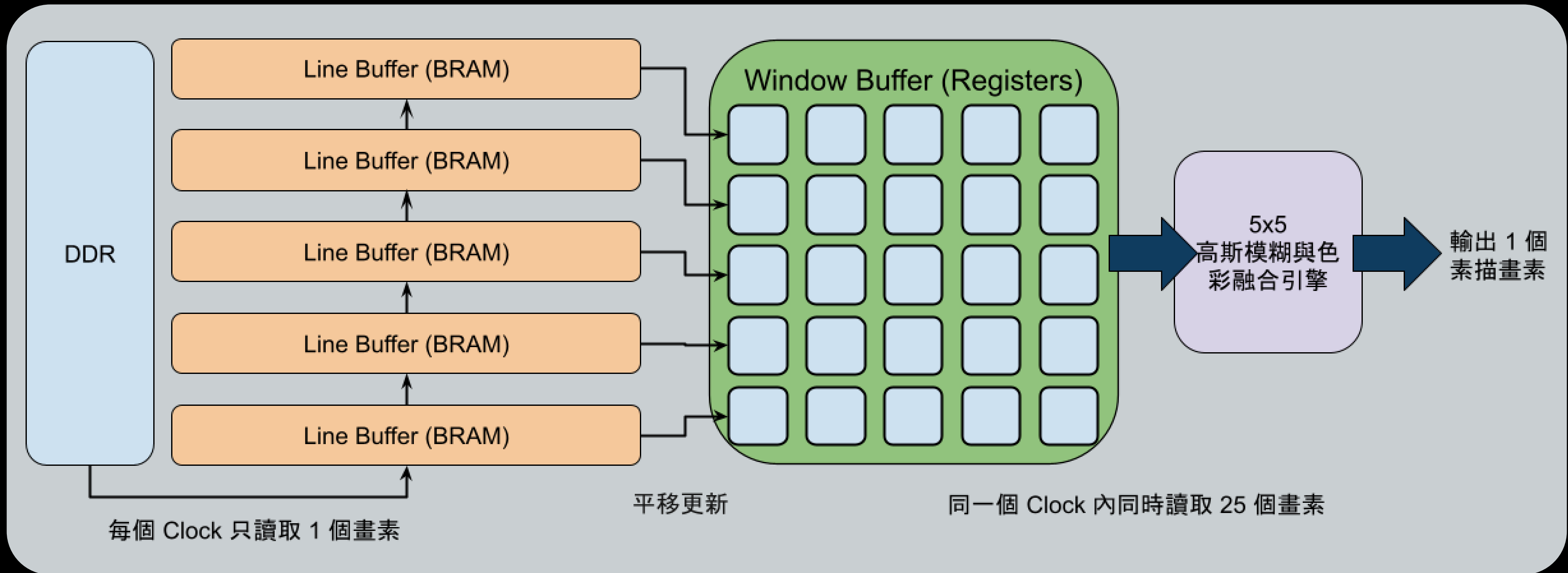
記憶體階層優化 (Memory Hierarchy Optimization) :

針對影像卷積之空間局部性 (Spatial Locality)，客製化 5×5 之 Line Buffer 與 Window Buffer 移位暫存器結構，將外部記憶體存取次數降至最低。

零時延除法與防溢位設計：

於 Color Dodge 運算中，藉由將高斯權重總和設計為 256，以位元右移 ($\gg 8$) 取代硬體除法器；並於分母加入常數補償 ($B+1$) 以確保系統穩定性 (避免除以0)。

硬體微架構設計：記憶體階層優化 (Line & Window Buffer)



硬體微架構設計與最佳化 (Sobel Edge Detection)

管線化處理 (Pipelining) : Vitis HLS 合成報告如下，證明系統成功達成 Initiation Interval (II) = 1 之理想吞吐量。

Synthesis Summary Report of 'sketch_filter'

General Information

Date:Sun Jun 14 23:45:25 2026

Version: 2022.2 (Build 3670227 on Oct 13 2022)

Project: hls_final_project

Solution:solution1 (Vivado IP Flow Target)

Product family:zynq

Target device:xc7z020-clg400-1

Timing Estimate

Target	Estimated	Uncertainty	
10.00 ns	7.300 ns	2.70 ns	

Performance & Resource Estimates ⓘ

Modules

Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
⌵ sketch_filter				-	-	-	-	-	-	no	2	6	3152	4580	0
⌵ sketch_filter_Pipeline_VITIS_LOOP_32_1_VITIS_LOOP_33_2				-	-	-	-	-	-	no	2	0	691	802	0

Performance Pragma

Modules & Loops	Target TI(cycles)	TI(cycles)	TI met
⌵ sketch_filter		-	-
⌵ sketch_filter_Pipeline_VITIS_LOOP_32_1_VITIS_LOOP_33_2		-	-

硬體微架構設計與最佳化 (Sobel Edge Detection)

管線化處理 (Pipelining)：Vitis HLS 合成報告如下，證明系統成功達成 Initiation Interval (II) = 1 之理想吞吐量。

Cosimulation Report for 'sketch_filter'

General Information

Date:	Tue Jun 16 21:07:16 2026	Solution:	solution1 (Vivado IP Flow Target)
Version:	2022.2 (Build 3670227 on Oct 13 2022)	Product family:	zynq
Project:	hls_final_project	Target device:	xc7z020-clg400-1
Status:	Pass		

Cosim Options

Tool: Vivado XSIM	RTL: Verilog
-------------------	--------------

Performance Estimates



Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
sketch_filter				2073648	2073648	2073648
sketch_filter_Pipeline_VITIS_LOOP_32_1_VITIS_LOOP_33_2				2073609	2073609	2073609

硬體微架構設計與最佳化 (Gaussian Blur & Color Dodge)

管線化處理 (Pipelining) : Vitis HLS 合成報告如下，證明系統成功達成 Initiation Interval (II) = 1 之理想吞吐量。

Synthesis Summary Report of 'sketch_filter'

General Information

Date: Tue Jun 16 18:34:45 2026

Version: 2022.2 (Build 3670227 on Oct 13 2022)

Project: hls_final_gaussian

Solution: solution1 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg400-1

Timing Estimate

Target	Estimated	Uncertainty
10.00 ns	7.300 ns	2.70 ns

Performance & Resource Estimates ⓘ

Modules

Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
sketch_filter				-	-	-	-	-	-	no	4	6	5051	5774	0
sketch_filter_Pipeline_VITIS_LOOP_28_1_VITIS_LOOP_29_2				-	-	-	-	-	-	no	4	0	2590	1996	0

Performance Pragma

Modules & Loops	Target TI(cycles)	TI(cycles)	TI met
sketch_filter		-	-
sketch_filter_Pipeline_VITIS_LOOP_28_1_VITIS_LOOP_29_2		-	-

硬體微架構設計與最佳化 (Gaussian Blur & Color Dodge)

管線化處理 (Pipelining)：Vitis HLS 合成報告如下，證明系統成功達成 Initiation Interval (II) = 1 之理想吞吐量。

Cosimulation Report for 'sketch_filter'

General Information

Date: Tue Jun 16 21:30:55 2026

Version: 2022.2 (Build 3670227 on Oct 13 2022)

Project: hls_final_gaussian

Status: Pass

Solution: solution1 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg400-1

Cosim Options

Tool: Vivado XSIM

RTL: Verilog

Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
sketch_filter				2073671	2073671	2073671
sketch_filter_Pipeline_VITIS_LOOP_28_1_VITIS_LOOP_29_2				2073632	2073632	2073632

軟硬體協同設計與系統整合

AXI 匯流排架構設計：

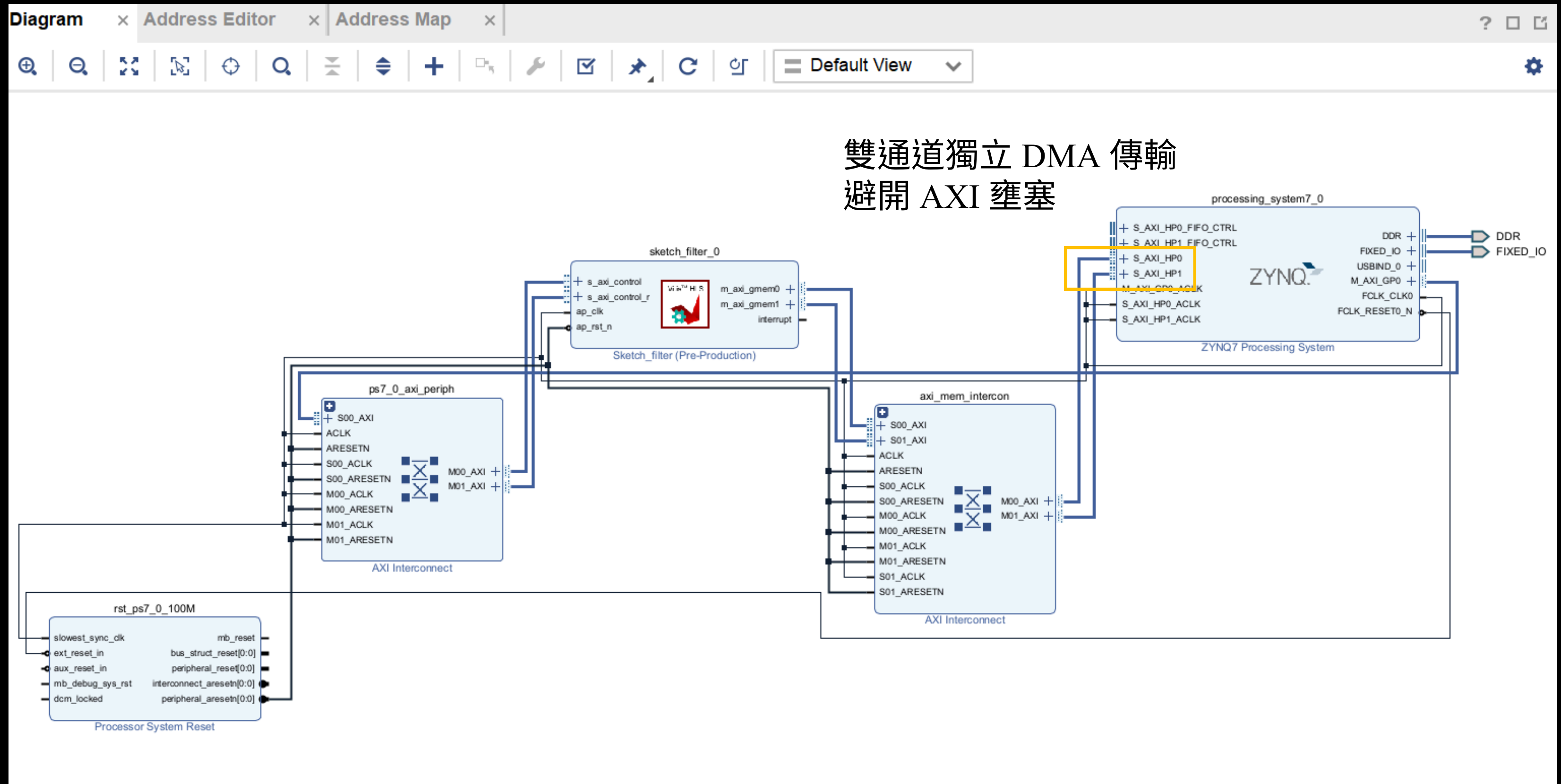
控制層：由 PS 端透過 AXI-Lite 介面配置影像解析度與控制訊號。

資料層：由 IP 內建之 AXI Master 介面主動存取 DDR 記憶體 (DMA 傳輸)。

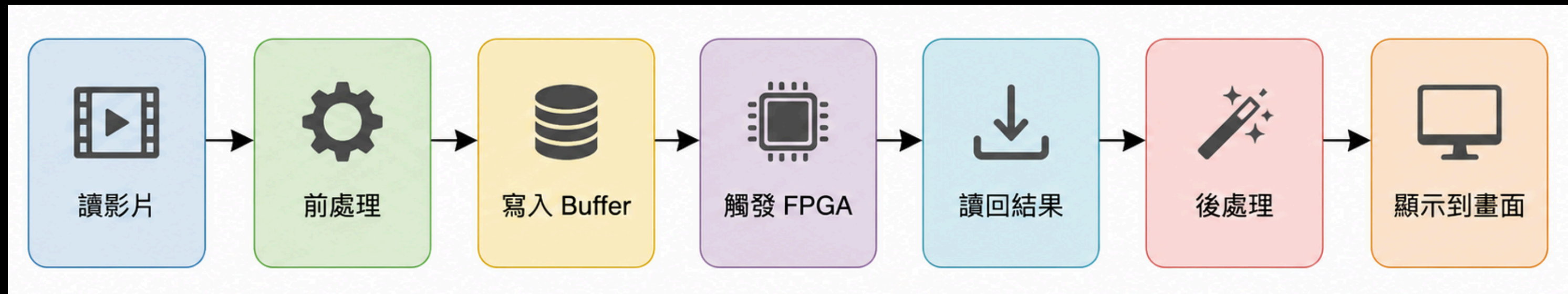
系統實作挑戰與對策：

在 Vivado Block Design 中，為避免 AXI Crossbar 之位址衝突 (Address Overlap)，採用雙通道獨立傳輸（HP0 負責讀取，HP1 負責寫入）以最大化記憶體頻寬。

Block Design



軟體架構總覽



整體執行緒：

主執行緒：UI元件 -> 調整參數

背景執行緒：影片處理

兩者透過 `current_settings` 溝通

目的：影片隨參數即時更新

```
try:
    while cap.isOpened() and current_settings['play_video']:
        ret, frame = cap.read()
        if not ret or frame is None:
            cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
            continue
        mode = current_settings['filter_type']
        c_gain = current_settings['contrast_gain']
        t_val = current_settings['threshold_val']
```

5. 啟動背景執行緒

```
t = threading.Thread(target=video_loop, daemon=True)
t.start()
```


PYNQ 初始化

1. 載入硬體與初始化

```
overlay = Overlay('./sketch_filter.bit')
ip_control = overlay.sketch_filter_0.s_axi_control
ip_memory = overlay.sketch_filter_0.s_axi_control_r

h, w = 720, 1280
in_buf = allocate(shape=(h, w), dtype=np.uint8)
out_buf = allocate(shape=(h, w), dtype=np.uint8)

ip_memory.write(0x10, in_buf.device_address)
ip_memory.write(0x1C, out_buf.device_address)
ip_control.write(0x10, h)
ip_control.write(0x18, w)
```

每幀處理流程

前處理(軟體做)：

```
color_resized = cv2.resize(frame, (w, h))
gray_resized  = cv2.cvtColor(color_resized, cv2.COLOR_BGR2GRAY)
alpha         = c_gain / 10.0
adjusted_input = cv2.convertScaleAbs(gray_resized, alpha=alpha, beta=0)
in_buf[:]     = adjusted_input[:]
```

觸發與等待（軟硬體介面）：

```
ip_control.write(0x00, 1)
while not (ip_control.read(0x00) & 0x2):
    pass
out_buf.invalidate()
fpga_raw = np.array(out_buf)
```


UI 設計

```
dropdown_filter = widgets Dropdown(
    options=['1. FPGA 原生灰階邊緣', '2. 銳化黑白鉛筆素描', '3. 彩色鉛筆插畫融合'],
    value='3. 彩色鉛筆插畫融合',
    description='濾鏡選擇:'
)
slider_contrast = widgets.IntSlider(min=5, max=25, step=1, value=10, description='對比(強度):')
slider_thresh = widgets.IntSlider(min=50, max=250, step=5, value=180, description='線條細節:')
btn_stop = widgets.Button(description="🛑 停止影片", button_style='danger')

def on_ui_change(change):
    current_settings['filter_type'] = dropdown_filter.value
    current_settings['contrast_gain'] = slider_contrast.value
    current_settings['threshold_val'] = slider_thresh.value

dropdown_filter.observe(on_ui_change, names='value')
slider_contrast.observe(on_ui_change, names='value')
slider_thresh.observe(on_ui_change, names='value')
```

三種濾鏡後處理

FPGA 原生(素描)：

```
if mode == '1. FPGA 原生灰階邊緣':  
    final_frame = fpga_raw
```

漫畫：

```
elif mode == '2. 銳化黑白漫畫':  
    _, binary_sketch = cv2.threshold(fpga_raw, t_val, 255, cv2.THRESH_BINARY)  
    final_frame = binary_sketch
```

彩色插畫：

```
elif mode == '3. 彩色鉛筆插畫融合':  
    _, binary_sketch = cv2.threshold(fpga_raw, t_val, 255, cv2.THRESH_BINARY)  
    sketch_3ch = cv2.merge([binary_sketch, binary_sketch, binary_sketch])  
    final_frame = cv2.divide(color_resized, sketch_3ch, scale=256)
```

實機測試功能展示 (PYNQ Demo)

圖片處理成功！大小：(720, 1280)
正在啟動 FPGA 硬體加速運算...
FPGA 運算完成！純硬體加速耗時：9.93 ms

Original Gray



FPGA Sketch Result





類素描

濾鏡選擇: 1. FPGA Raw Edge (基礎版) ▾

亮度加減: 0

線條細節: 180

Mode: 1. FPGA Raw Edge (□□□) | Brightness□: 0 | □□: 180



濾鏡選擇: 2. Binary Pencil Sketch (銳化素 ▾

亮度加減: 0

線條細節: 180

Mode: 2. Binary Pencil Sketch (□□□□) | Brightness□: 0 | □□: 180



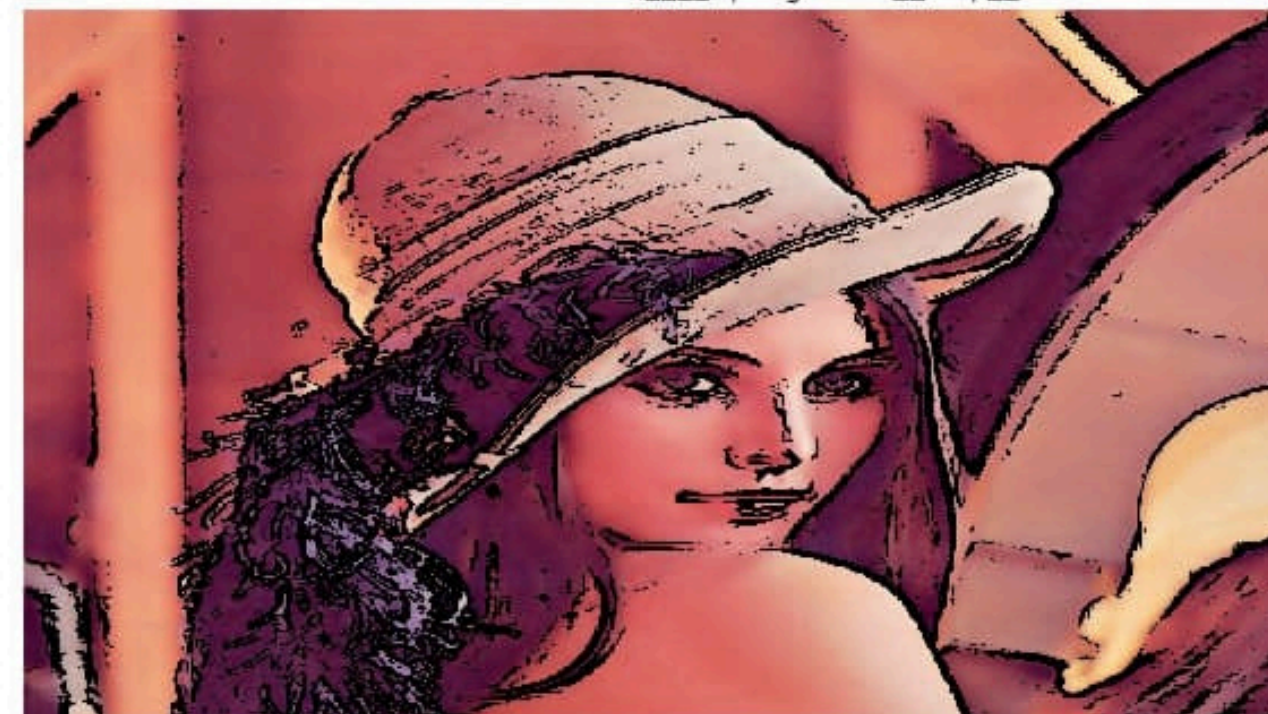
類漫畫

濾鏡選擇: 3. Combined Pencil Art (色彩融 ▾

亮度加減: 0

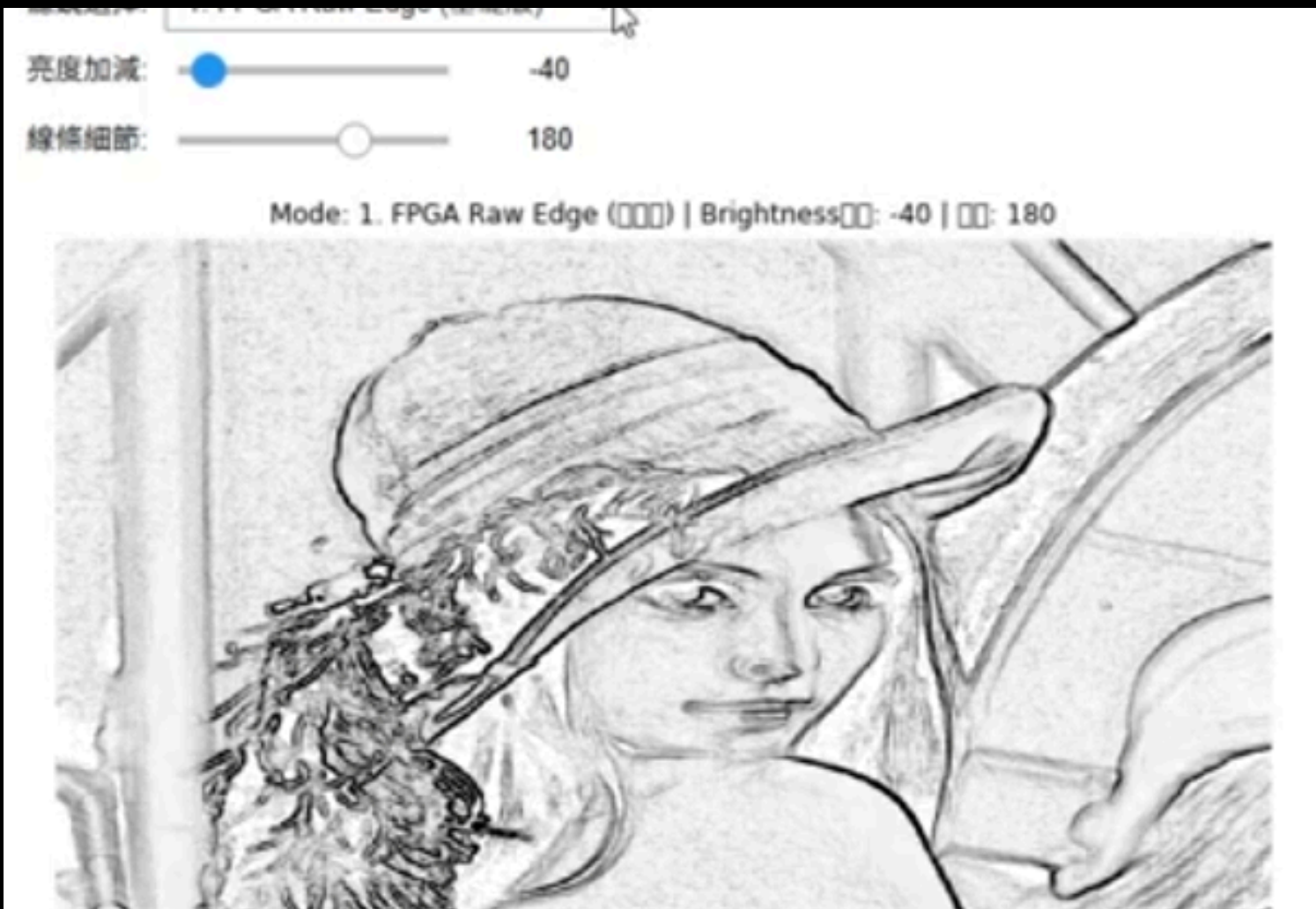
線條細節: 180

Mode: 3. Combined Pencil Art (□□□□) | Brightness□: 0 | □□: 180



類動畫

亮度對比



線條對比



實機測試成果展示 (PYNQ Demo)

影片